



山东大学

## 信息科学与工程学院

2024—2025 学年第二学期

# 实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 3.1 子程序实验

专 业 班 级 通信工程 三班

学 生 学 号                     

学 生 姓 名                     

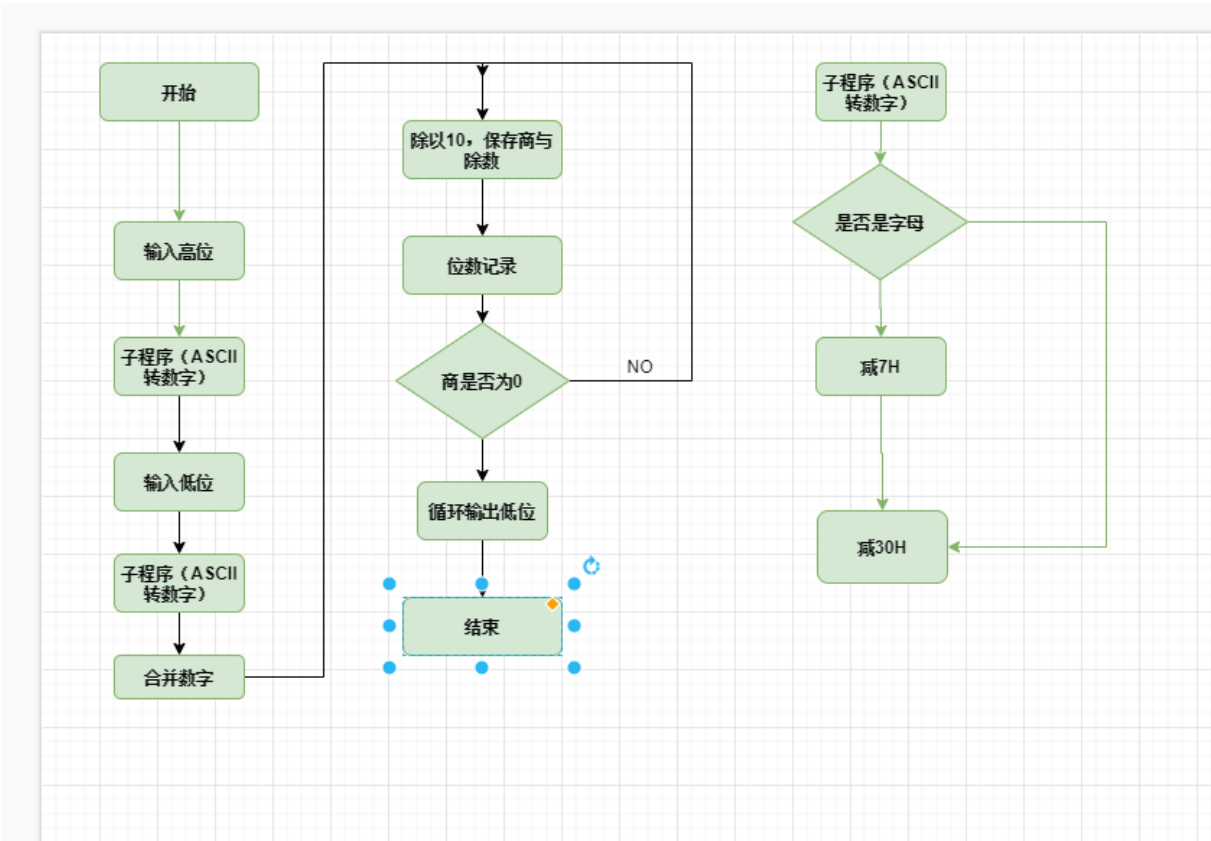
实 验 时 间 2025 年 3 月 14 日

# 实验报告

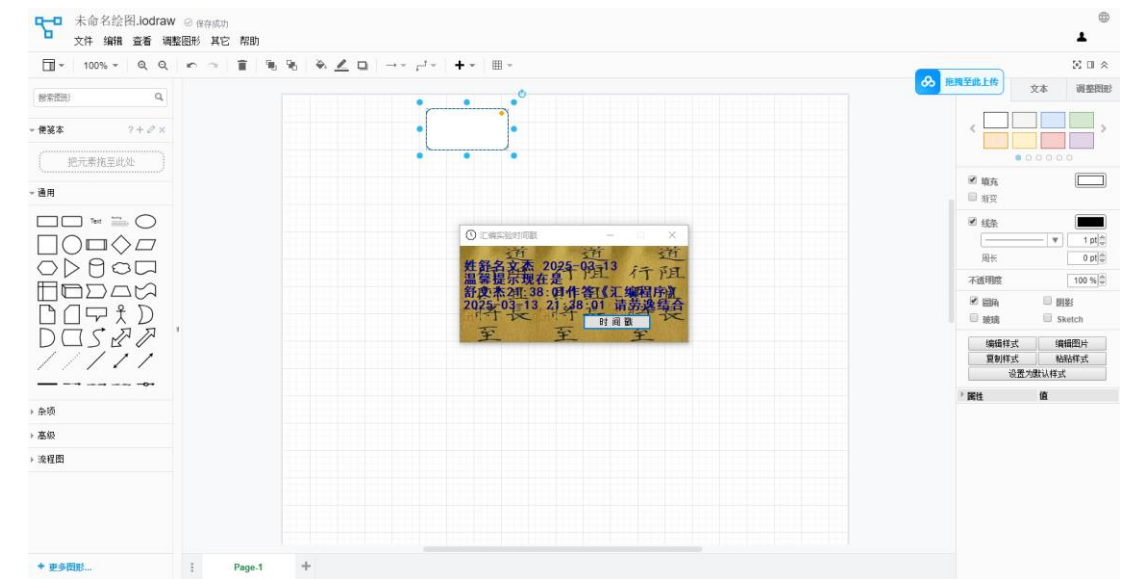
## 一、 实验内容

2.将键盘上输入的十六进制数转换成十进制数，并在屏幕上显示。（编写程序，详细注释并画程序流程图）

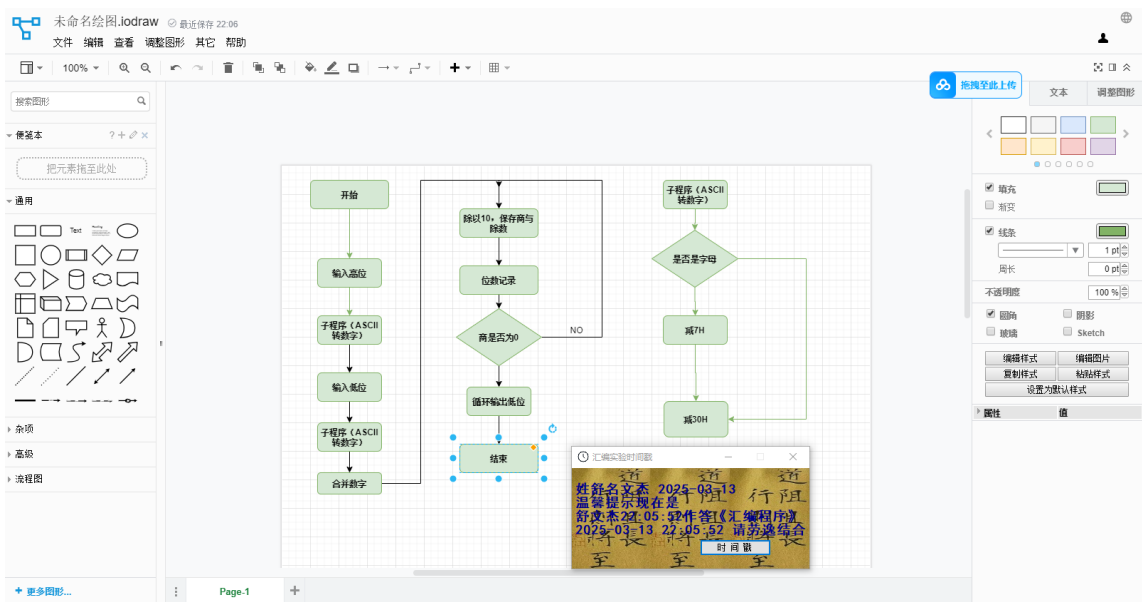
画出流程图如下：



图表 1



图表 2



图表 3

源代码如下：

1. DATAS SEGMENT
2. BUF DB 20 ;缓冲区大小, 预留 20 字节
3. DB ? ;用于存储实际输入的字符数
4. DB 20 DUP(?) ;存储输入的 20 个字符
5. DATAS ENDS

6.

7. STACKS SEGMENT STACK

8. DB 200 DUP(?) ; 堆栈段, 分配 200 字节空间

9. STACKS ENDS

10.

11. CODES SEGMENT

12. ASSUME CS:CODES, DS:DATAS, SS:STACKS

13.

14. START:

15. MOV AX, DATAS ; 初始化数据段寄存器 DS

16. MOV DS, AX

17. MOV DX, 0000H ; 清空 DX 寄存器, 可能用于后续功能

18.

19. ; 读取第一个字符

20. MOV AH, 01H ; DOS 功能号 01H, 读取键盘输入字符 (回显)

21. INT 21H ; 调用 DOS 中断, 输入字符存储在 AL 中

22. CALL TRANS ; 调用转换过程, 将 ASCII 字符转为十六进制数值

23. PUSH AX ; 将转换后的数值暂存到堆栈

24.

25. ; 读取第二个字符

26. MOV AH, 01H ; 再次读取字符

27. INT 21H

```

28.  POP BX      ; 取出第一个字符的数值到 BX

29.  CALL TRANS   ; 转换第二个字符

30.  MOV CL, 4     ; 将第一个字符数值左移 4 位（高四位）

31.  SHL BL, CL    ; 例如: 0xA -> 0xA0

32.  ADD AX, BX    ; 合并两个字符的数值（如 0xA0 + 0x3 = 0xA3）

33.  PUSH AX      ; 保存合并后的十六进制数值

34.

35.  ; 将十六进制转换为十进制并输出

36.  POP AX       ; 取出合并后的数值到 AX

37.  MOV CX, 0000H ; CX 记录十进制位数

38.  MOV BX, 000AH ; BX 作为除数 10

39.

40. DIV_10:       ; 循环除以 10，分解十进制各位

41.  XOR DX, DX    ; 清零 DX（因 DIV 使用 DX:AX 作为被除数）

42.  DIV BX        ; AX = 商，DX = 余数（当前十进制位）

43.  PUSH DX       ; 余数压栈保存（逆序存储）

44.  INC CX        ; 位数加 1

45.  CMP AX, 0     ; 检查商是否为 0

46.  JNE DIV_10    ; 不为 0 则继续循环

47.

48.  ; 输出回车符

49.  MOV DL, 0DH   ; 准备输出回车符（ASCII 0x0D）
    
```

```

50.  MOV AH, 02H    ; DOS 功能号 02H, 显示输出
51.  INT 21H
52.
53. Print_loop:      ; 循环输出十进制各位
54.  POP DX          ; 弹出十进制位（从最高位开始）
55.  ADD DL, 30H      ; 转换为 ASCII 字符（如 3 -> '3'）
56.  MOV AH, 02H      ; 显示该字符
57.  INT 21H
58.  LOOP Print_loop ; 循环 CX 次，输出所有位
59.
60.  ; 程序结束
61.  MOV AH, 4CH      ; DOS 功能号 4CH, 退出程序
62.  INT 21H
63.
64. ; 子程序：将 ASCII 字符转换为十六进制数值
65. TRANS PROC
66.  MOV AH, 00H      ; 清零 AH, 确保 AX 高 8 位为 0
67.  CMP AL, 39H      ; 比较字符是否大于 '9'
68.  JBE NEXT2        ; 若为 '0'-'9', 跳转到 NEXT2
69.  SUB AL, 07H      ; 若为 'A'-'F', 先减 7（如 'A' -> 0x41-7=0x3A）
70. NEXT2:
71.  SUB AL, 30H      ; 转换为数值（'0'-'9'减 0x30, 'A'-'F'减 0x30 后为 0xA-0xF）

```

72. RET ; 返回, 结果在 AL 中

73. TRANS ENDP

74.

75. CODES ENDS

76. **END START**

## 二、 实验步骤



```

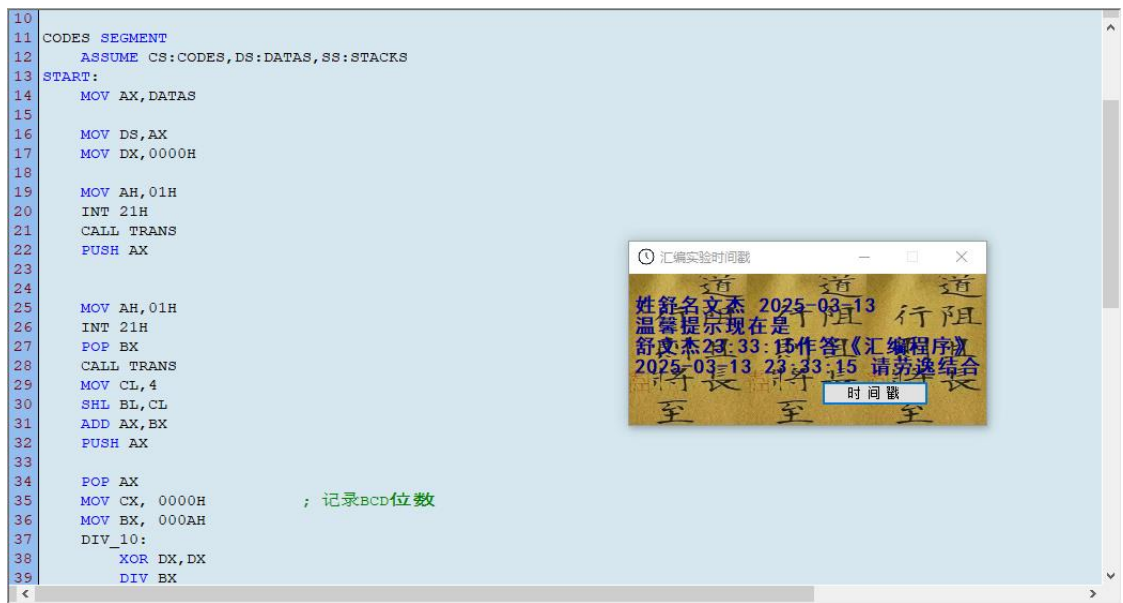
01 ;默认采用ML6.11汇编程序
02 DATAS SEGMENT
03 ;此处输入数据段代码
04 DATAS ENDS
05
06 STACKS SEGMENT
07 ;此处输入堆栈段代码
08 STACKS ENDS
09
10 CODES SEGMENT
11 ASSUME CS:CODES,DS:DATAS,SS:STACKS
12 START:
13     MOV AX,DATAS
14     MOV DS,AX
15     ;此处输入代码段代码
16     MOV AH,4CH
17     INT 21H
18 CODES ENDS
19     END START
    
```

汇编实验时间戳

姓名: 文杰 2025-03-13 11:26  
温馨提示: 现在是 2025-03-13 22:11:26 请抓紧时间  
时间戳

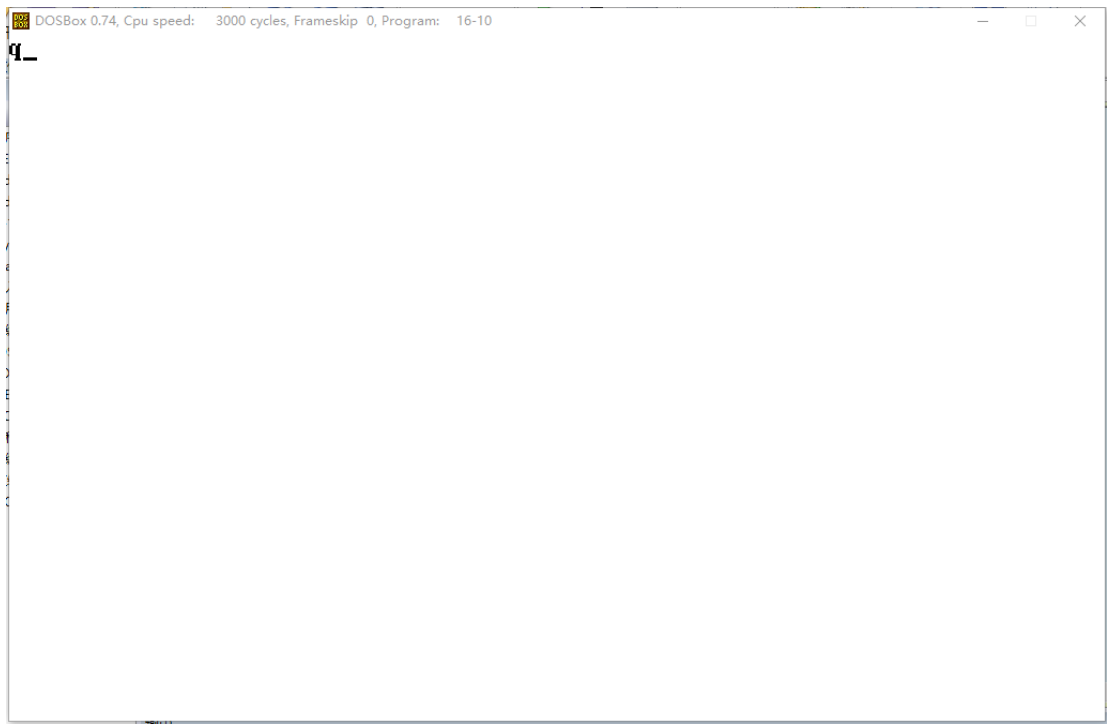
图表 4

1、用 windows 集成编译环境编写. ASM 源程序。



图表 5

2、运行测试代码。



只能输入一个数，没有任何反应，这是为什么呢？

```

46 ADD DL,30H
47 MOV AH, 02H
48 INT 21H
49
50 POP BX
51 ADD BL,30H
52 MOV AH, 02H
53 INT 21H
54
55 MOV AH,4CH
56 INT 21H
57
58 TRANS10 PROC
59 POP AX
60
61 ;PUSH BX
62
63 POP AX
64 CMP AX, '9'
65 JBE NEXT0
66 SUB AL,07H
67 NEXT0:
68 ADD AL,30H
69 AAA
70 MOV BX,AX
71 PUSH BX
72 RET
73 TRANS10 ENDP
74
75 CODES ENDS
76 END START
77

```

原来是我把标志位搞没了

于是，我使用栈指针来精确移动数据：

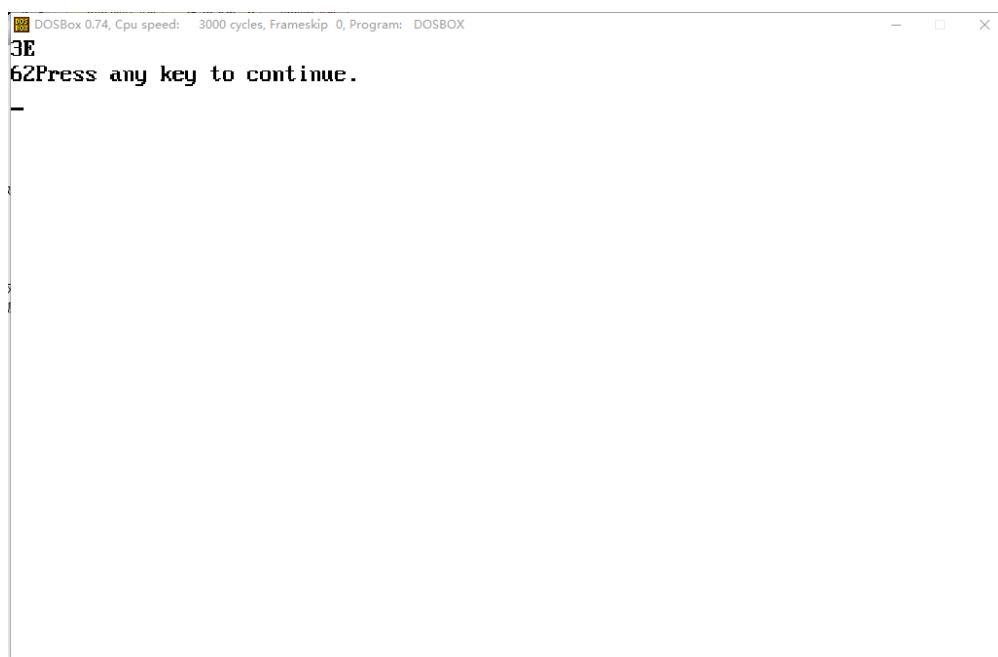
```

52 INT 21H
53
54 MOV AH,4CH
55 INT 21H
56
57 TRANS10 PROC
58 PUSH BP
59 MOV BP, SP
60 MOV AX, [BP+4]
61 CMP AX, '9'
62 JBE NEXT0
63 SUB AL,07H
64 NEXT0:
65 ADD AL,30H
66 AAA
67 MOV [BP+4],AX
68 POP BP
69 RET
70 TRANS10 ENDP
71
72 CODES ENDS
73 END START
74
75

```

利用栈指针来保存

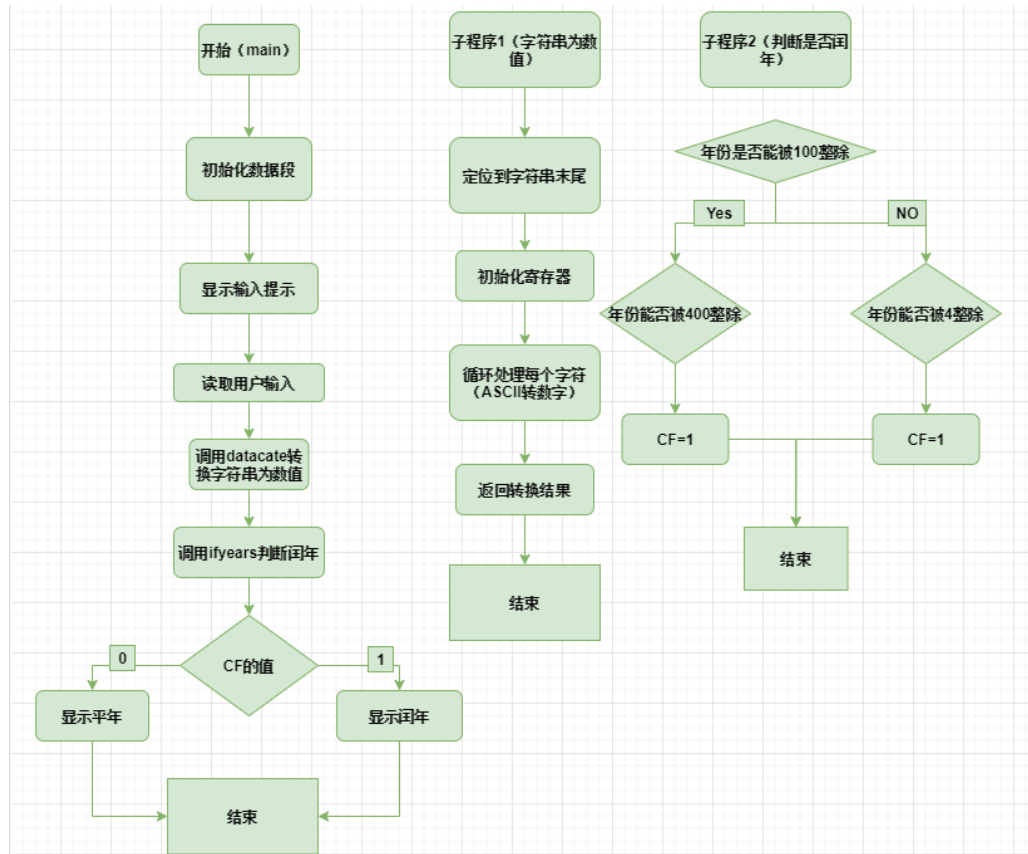
重复运行代码：



实验成功

2.判断闰年程序学习:

画出流程图:



源代码注释如下：

1. data segment
2. `infor db 0dh,0ah,'Please input a year: $'` ; 换行+输入年份提示, \$为字符串结束符
3. `Y db 0dh,0ah,'This is a leap year! $'` ; 换行+闰年提示
4. `N db 0dh,0ah,'This is not a leap year! $'` ; 换行+非闰年提示
5. `w dw 0` ; 存储转换后的年份数值 (字类型)
6. `buf db 8` ; DOS 输入缓冲区结构:
7. `db ?` ; 第一字节存储允许输入的最大字符数
8. `db 8 dup(?)` ; 第二字节存储实际输入字符数, 后续为字符存储区
9. data ends
- 10.

12. `db 200 dup(0)` ; 定义 200 字节的堆栈空间，初始化为 0

16. `assume ds:data,ss:stack,cs:code` ; 声明段寄存器与各段的关联

```
18.  mov ax,data
```

20. `lea dx,infon` ;加载输入提示信息的偏移地址到 DX

22. `int 21h` ;调用 DOS 中断显示提示信息

24.   mov ah,0Ah                   ; DOS 功能号 0AH, 缓冲输入

26. `mov cl,[buf+1]` ; 获取实际输入字符数到 CL

28. `lea di,buf+2` ; DI 指向输入的第一个字符的位置

30.   call ifyears               ; 调用闰年判断子程序

32. `lea dx,N` ; 加载非闰年提示信息的偏移地址

```

33.  mov ah,9

34.  int 21h          ; 显示非闰年提示

35.  jmp exit         ; 跳转到程序结束

36. a1:

37.  lea dx,Y          ; 加载闰年提示信息的偏移地址

38.  mov ah,9

39.  int 21h          ; 显示闰年提示

40. exit:

41.  mov ah,4ch        ; DOS 功能号 4CH，程序终止

42.  int 21h          ; 返回 DOS 系统

43.

44. ; 字符串转数值子程序（将 ASCII 字符串转换为二进制数值）

45. datacate proc near

46.  push cx           ; 保存 CX（字符数）

47.  dec cx            ; CX 减 1（因循环从 0 开始）

48.  lea si,buf+2      ; SI 指向输入的第一个字符

49. tt1:

50.  inc si            ; SI 递增，移动到下一个字符

51.  loop tt1          ; 循环直到 SI 指向字符串末尾

52.  pop cx            ; 恢复 CX（原始字符数）

53.  mov dh,30h        ; '0'的 ASCII 码，用于转换

54.  mov bl,10         ; 十进制基数为 10
    
```

```

55.  mov ax,1          ; 初始化权值为 1（个位）

56. l1:

57.  push ax           ; 保存当前权值

58.  push bx           ; 保存基数

59.  push dx           ; 保存'0'的 ASCII 码

60.  sub byte ptr [si],dh ; 当前字符 ASCII 码转数字（减去'0'）

61.  mov bl, byte ptr [si] ; 将转换后的数字存入 BL

62.  mov bh,0          ; 清空 BH，使 BX 为当前数字

63.  mul bx            ; AX = 权值（AX）* 当前数字（BX）

64.  add [w],ax         ; 将结果累加到年份变量 w

65.  pop dx            ; 恢复 DX（'0'的 ASCII 码）

66.  pop bx            ; 恢复 BX（基数）

67.  pop ax            ; 恢复 AX（当前权值）

68.  mul bl            ; AX = 权值 * 10（准备下一位）

69.  dec si            ; SI 递减，指向前一个字符

70.  loop l1           ; 循环处理所有字符

71.  ret              ; 返回主程序

72. datacate endp

73.

74. ; 闰年判断子程序（根据年份设置进位标志 CF）

75. ifyears proc near

76.  push bx           ; 保存 BX
    
```

```

77.  push cx          ; 保存 CX
78.  push dx          ; 保存 DX
79.  mov ax,[w]        ; 将年份数值加载到 AX
80.  mov cx,ax         ; 备份年份到 CX
81.  mov dx,0          ; 清空 DX（用于除法）
82.  mov bx,100        ; 检查能否被 100 整除
83.  div bx            ; AX=商，DX=余数（年份/100）
84.  cmp dx,0          ; 比较余数是否为 0
85.  jnz lab1          ; 余数非 0 跳转到 lab1（非整百年）
86.  mov ax,cx         ; 恢复年份到 AX（整百年情况）
87.  mov bx,400        ; 检查能否被 400 整除
88.  div bx
89.  cmp dx,0          ; 余数是否为 0
90.  jz lab2           ; 是则跳转到 lab2（闰年）
91.  cld               ; 清除进位标志（非闰年）
92.  jmp lab3
93. lab1:
94.  mov ax,cx         ; 恢复年份到 AX（非整百年）
95.  mov dx,0
96.  mov bx,4          ; 检查能否被 4 整除
97.  div bx
98.  cmp dx,0          ; 余数是否为 0
    
```

```

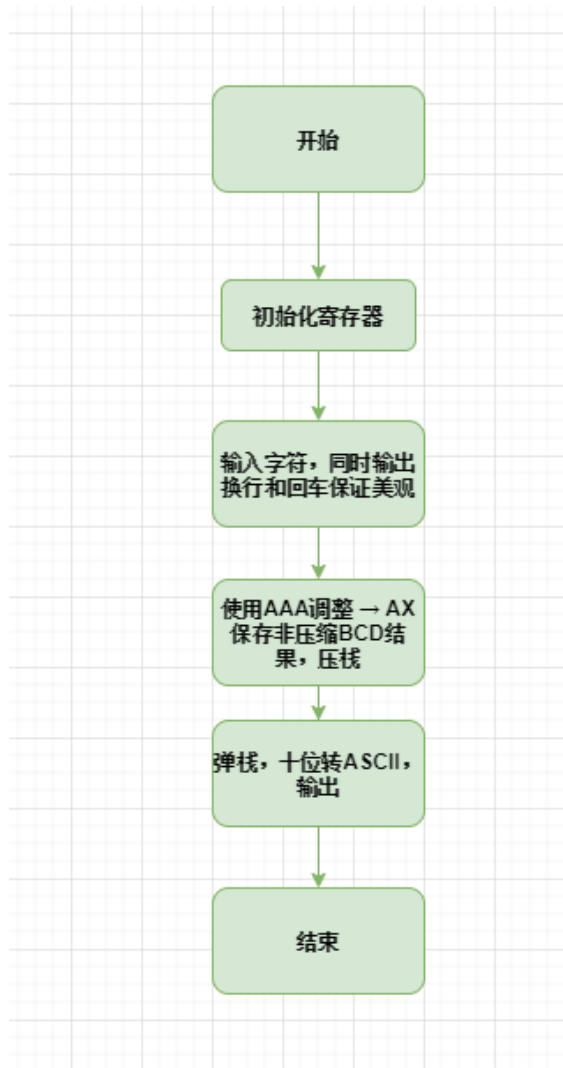
99.  jz lab2          ; 是则跳转到 lab2 (闰年)
100.  clc             ; 清除进位标志 (非闰年)
101.  jmp lab3
102. lab2:
103.  stc             ; 设置进位标志 CF=1 (闰年)
104. lab3:
105.  pop dx          ; 恢复 DX
106.  pop cx          ; 恢复 CX
107.  pop bx          ; 恢复 BX
108.  ret             ; 返回主程序
109. ifyears endp
110.
111. code ends
112. end start

```

### 3. 汇编实例学习和改进：两位数加法

(1) 实现输入两个变量，然后完成加法。

画出流程图



图表 6

源代码如下：

1. STACKS SEGMENT
2. DB 200 DUP(?) ; 定义堆栈段，预留 200 字节未初始化空间
3. STACKS ENDS
- 4.
5. DATAS SEGMENT
6. ; 数据段定义（此处为空）
7. DATAS ENDS

8.

9. CODES SEGMENT

10. ASSUME CS:CODES, DS:DATAS, SS:STACKS ; 设置段寄存器关联

11.

12. **START:**

13. ; 初始化数据段

14. MOV AX, DATAS

15. MOV DS, AX

16.

17. ; 读取第一个字符输入（结果存入 AL）

18. MOV AH, 01H ; DOS 功能号 01H（带回显的字符输入）

19. **INT 21H** ; 执行中断

20. MOV BL, AL ; 将输入字符保存到 BL 寄存器

21.

22. ; 输出回车换行（光标移动到新行首）

23. MOV DL, 0DH ; 回车符 ASCII 码

24. MOV AH, 02H ; DOS 功能号 02H（字符输出）

25. **INT 21H**

26. MOV DL, 0AH ; 换行符 ASCII 码

27. MOV AH, 02H

28. **INT 21H**

29.

```

30.    ; 读取第二个字符输入

31.    MOV AH, 01H

32.    INT 21H    ; 输入结果存入 AL

33.    MOV AH, 00H    ; 清零 AH, 准备加法运算

34.    ADD AL, BL    ; 将两个字符的 ASCII 码相加 (存在逻辑错误)

35.    AAA          ; ASCII 加法调整, 将结果转为非压缩 BCD 格式

36.

37.    PUSH AX      ; 将结果 (AH=十位, AL=个位) 压栈保存

38.

39.    ; 再次输出回车换行

40.    MOV DL, 0DH

41.    MOV AH, 02H

42.    INT 21H

43.    MOV DL, 0AH

44.    MOV AH, 02H

45.    INT 21H

46.

47.    ; 输出计算结果

48.    POP AX       ; 弹出结果到 AX

49.    MOV BX, AX    ; 临时保存到 BX

50.    PUSH BX       ; 重新压栈保存

51.

```

```

52.    ; 输出十位数 (AH 部分)

53.    MOV DL, AH    ; 将十位数值移到 DL

54.    ADD DL, 30H    ; 转为 ASCII 字符

55.    MOV AH, 02H

56.    INT 21H

57.

58.    ; 输出个位数 (AL 部分)

59.    POP AX        ; 恢复结果到 AX

60.    MOV DL, AL

61.    ADD DL, 30H    ; 转为 ASCII 字符

62.    MOV AH, 02H

63.    INT 21H

64.

65.    ; 程序终止

66.    MOV AH, 4CH    ; DOS 功能号 4CH (程序退出)

67.    INT 21H

68.

69. CODES ENDS

70. END START

```

## 二. 实验步骤

1、用 windows 集成编译环境编写 ASM 源程序。

在有着多次输入编程经验后，我很快完成了实验：

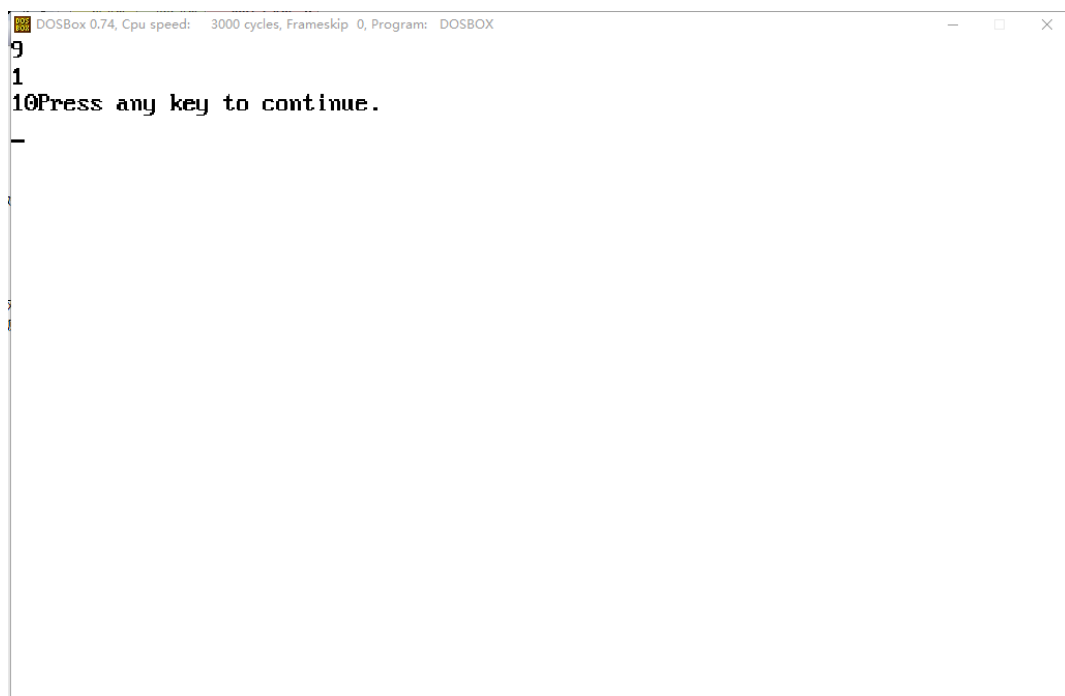
```

01 STACKS SEGMENT
02     DB 200 DUP(?)
03 STACKS ENDS
04
05 DATAS SEGMENT
06     ;此处输入数据段代码
07 DATAS ENDS
08
09 CODES SEGMENT
10     ASSUME CS:CODES,DS:DATAS,SS:STACKS
11 START:
12     MOV AX,DATAS
13     MOV DS,AX
14
15     MOV AH,01H
16     INT 21H
17     MOV BL,AL
18
19     MOV DL, 0DH ; 回车符
20     MOV AH, 02H
21     INT 21H
22
23     MOV DL, 0AH ; 换行符
24     MOV AH, 02H
25     INT 21H
26
27     MOV AH,01H
28     INT 21H
29     MOV AH,00H
30     ADD AL,BL

```

图表 9

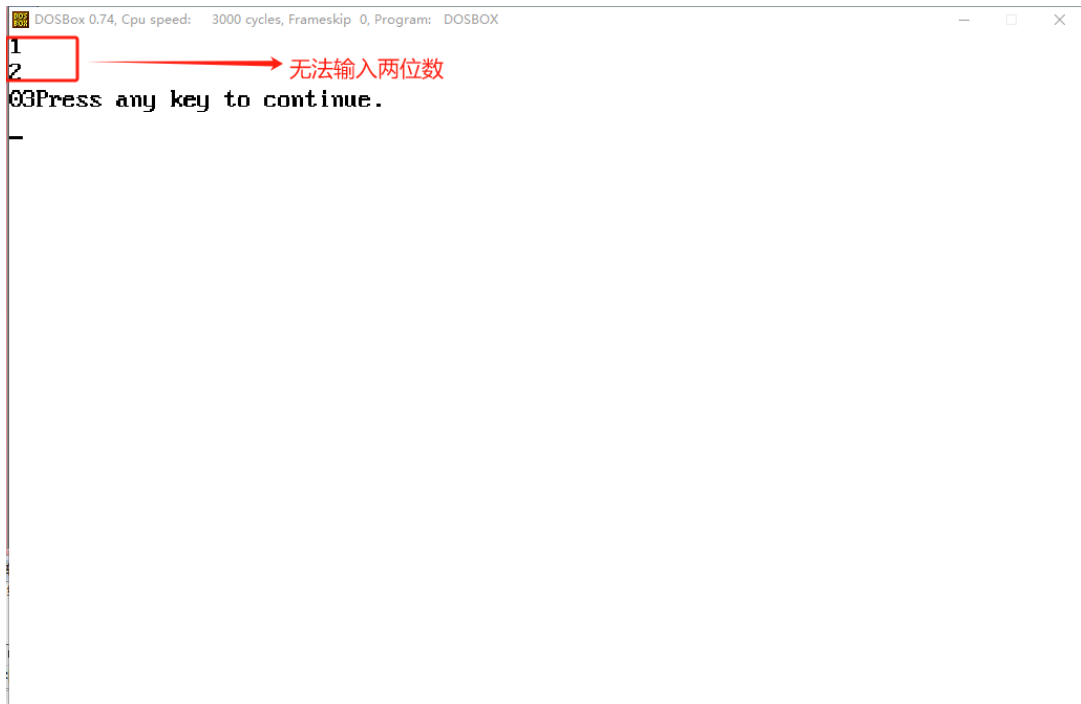
## 2.运行测试一下:



图表 10

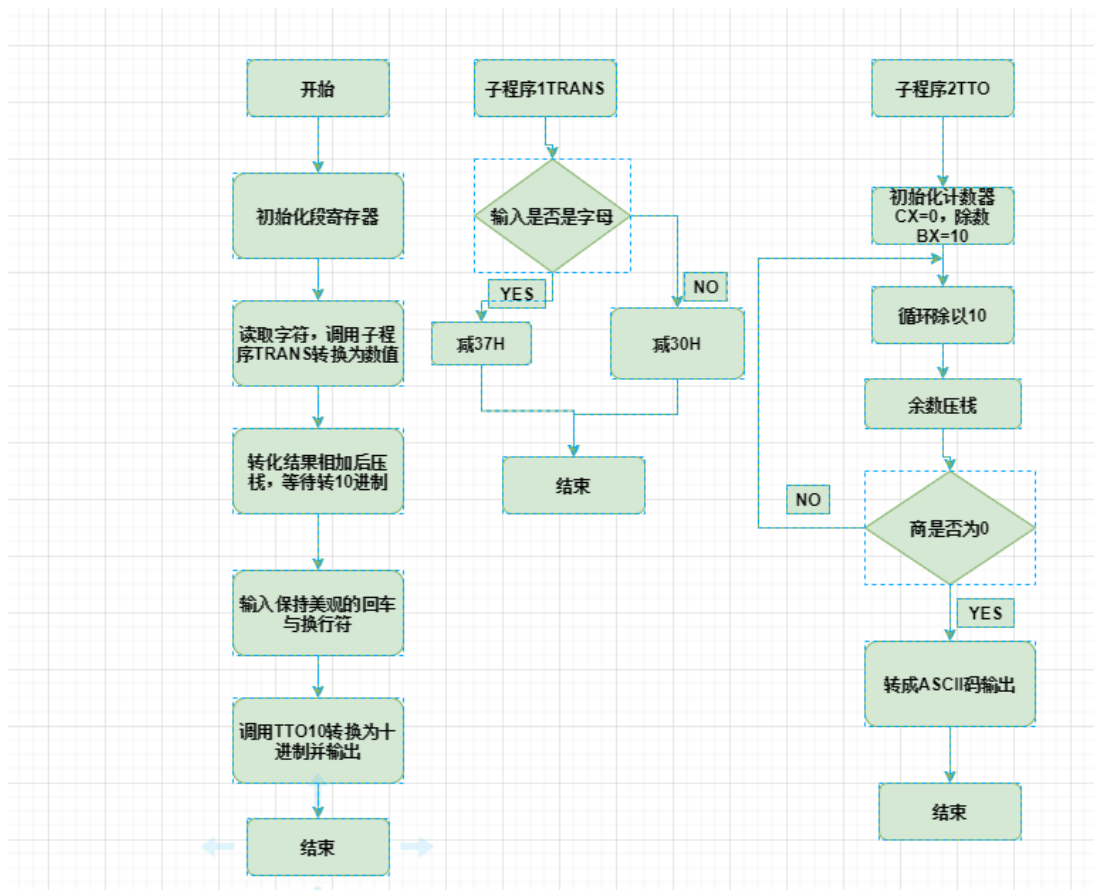
可以看出，完美运行，一次过

但是，如果我输入两位数，则会出现：



图表 11

这是因为我没有考虑输入两位数，同时，它也会多输出一个 0，那我再修改一下：  
 2. 处理输入两位数的溢出，利用我们前面刚刚完成的16进制转换10进制并显示的程序，在这里用子程序来调用。  
 流程图如下：



图表 12

源代码如下

1. DATAS SEGMENT

2. ; 数据段定义 (此处未使用具体数据)

3. DATAS ENDS

4.

5. STACKS SEGMENT

6. DB 200 DUP(?) ; 定义 200 字节的堆栈空间

7. STACKS ENDS

8.

9. CODES SEGMENT

10. ASSUME CS:CODES,DS:DATAS,SS:STACKS ; 设置段寄存器与段的关系

11. **START:**

12. `MOV AX,DATAS` ; 初始化数据段

13. `MOV DS,AX`

14.

15. ;

16. `MOV AH,01H` ; DOS 功能号 **01H**: 带回显的字符输入

17. **INT 21H** ; 执行输入, 结果保存在 AL

18. **CALL TRANS** ; 将 ASCII 字符转换为数值

19. `MOV BL,AL` ; 保存第一个数值到 BL

20.

21. ; 换行处理

22. `MOV DL, 0DH` ; 准备输出回车符

23. `MOV AH, 02H` ; DOS 功能号 **02H**: 字符输出

24. **INT 21H**

25. `MOV DL, 0AH` ; 准备输出换行符

26. `MOV AH, 02H`

27. **INT 21H**

28.

29. ;

30. `MOV AH,01H` ; 再次输入字符

31. **INT 21H**

32. **CALL TRANS** ; 转换为数值

```

33.  ADD AL,BL    ; 两个数值相加，结果在 AL
34.  PUSH AX      ; 保存结果到堆栈（AH 可能含垃圾值）
35.
36.  ; 再次换行
37.  MOV DL, 0DH
38.  MOV AH, 02H
39.  INT 21H
40.  MOV DL, 0AH
41.  MOV AH, 02H
42.  INT 21H
43.
44.  POP AX       ; 恢复结果到 AX（AL=和，AH 可能无效）
45.  CALL TTO10   ; 将 AL 中的二进制数转换为十进制输出
46.
47.  ; 程序终止
48.  MOV AH,4CH   ; DOS 功能号 4CH：带返回码终止
49.  INT 21H
50.
51.  ;
52.  TRANS PROC
53.  MOV AH,00H   ; 清空 AH（确保 AX 高字节为 0）
54.  CMP AL,39H   ; 比较输入字符与'9'

```

```

55.     JBE NEXT2    ;若是数字 (≤'9')，跳转
56.     SUB AL,07H   ;若是字母 (A-F)，调整 ASCII 差值
57.     NEXT2:
58.     SUB AL,30H   ;转换为实际数值 (0-15)
59.     RET          ;返回调用处
60.     TRANS ENDP
61.
62.     ;
63.     TTO10 PROC
64.     MOV CX, 0000H ;初始化计数器 (记录数字位数)
65.     MOV BX, 000AH ;除数 10 (用于十进制转换)
66.     DIV_10:
67.     XOR DX,DX     ;清空 DX (被除数高位)
68.     DIV BX        ;AX 除以 10，商在 AX，余数在 DX
69.     PUSH DX       ;保存余数 (当前位的数字)
70.     INC CX        ;位数计数器加 1
71.     CMP AX,0      ;检查商是否为 0
72.     JNE DIV_10    ;若不为 0，继续循环
73.
74.     Print_loop:
75.     POP DX         ;取出栈顶数字 (从高位到低位)
76.     ADD DL,30H    ;转换为 ASCII 字符

```

```

77.     MOV AH,02H    ;准备字符输出功能

78.     INT 21H       ;输出数字字符

79.     LOOP Print_loop ;循环直到所有位数输出完毕

80.     RET           ;返回调用处

81.     TIO10 ENDP

82.

83. CODES ENDS

84.     END START
    
```

## 1、用 windows 集成编译环境编写. ASM 源程序。

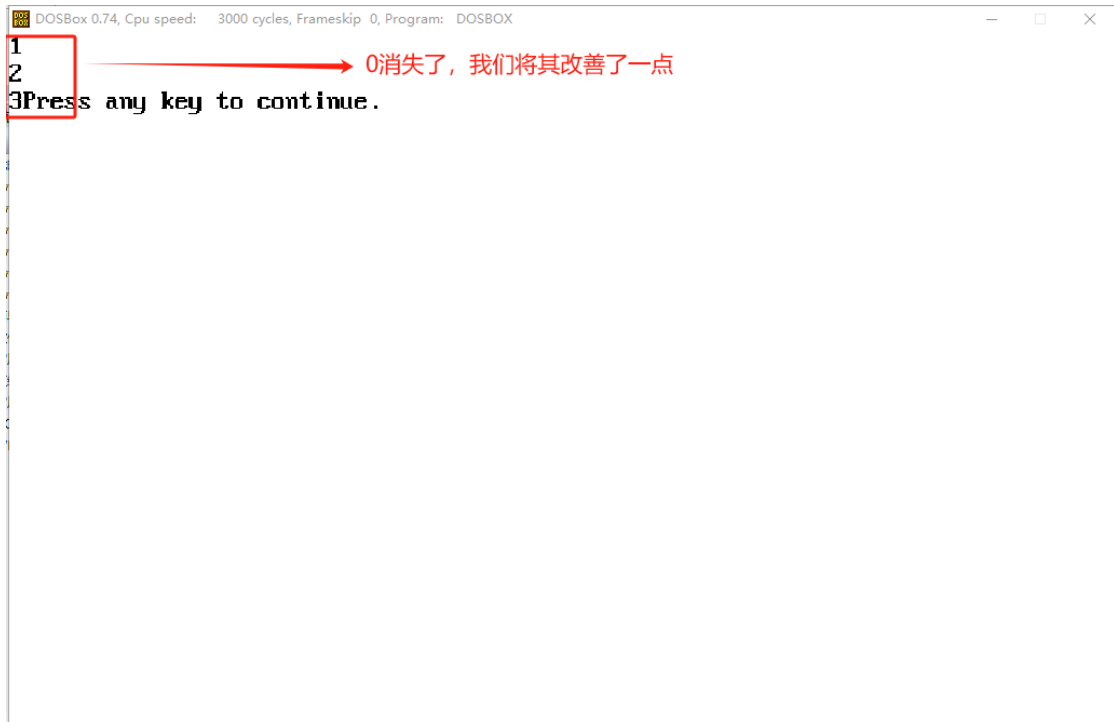


```

01 ;默认采用ML6.11汇编程序
02 DATAS SEGMENT
03 ;此处输入数据段代码
04 DATAS ENDS
05
06 STACKS SEGMENT
07     DB 200 DUP(?)
08 STACKS ENDS
09
10 CODES SEGMENT
11     ASSUME CS:CODES,DS:DATAS,SS:STACKS
12 START:
13     MOV AX,DATAS
14     MOV DS,AX
15
16     MOV AH,01H
17     INT 21H
18     CALL TRANS
19     MOV BL,AL
20
21     MOV DL, 0DH ; 回车符
22     MOV AH, 02H
23     INT 21H
24
25     MOV DL, 0AH ;换行符
26     MOV AH, 02H
27     INT 21H
28
29     MOV AH,01H
30     INT 21H
    
```

图表 13

## 2.我们可以运行一下，看看有没有什么 BUG。



图表 14



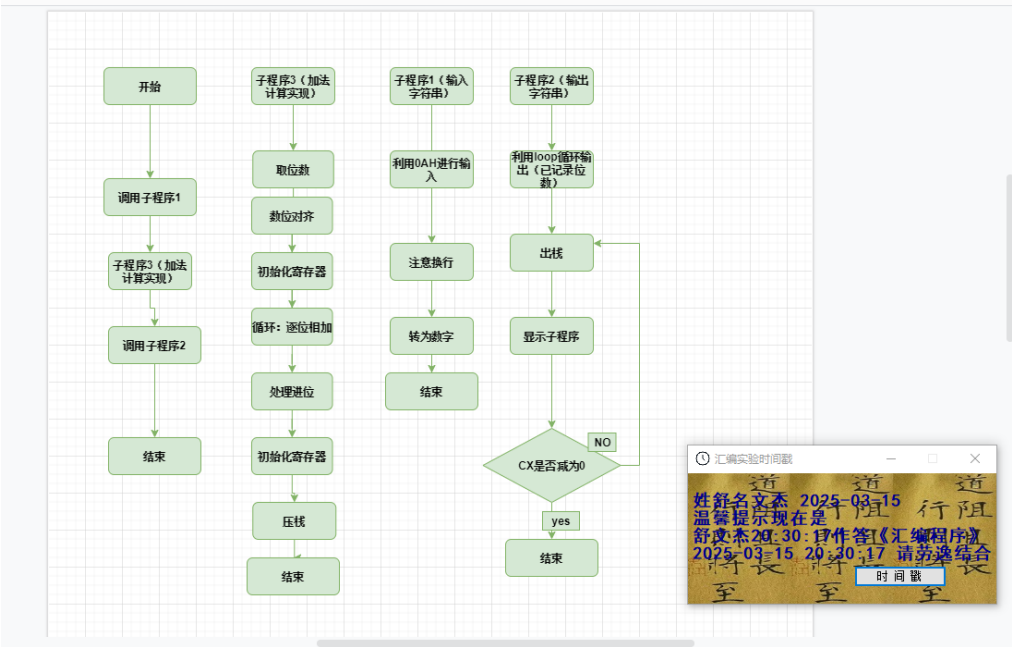
图表 15

可以发现，我们利用 16 进制间接输入了两位数，也解决了输出多出一个 0 的问题，但是，我们的输入的范围相当小，只有 0-15，我们还可以改进，在这里，我联想到了曾经学习过的高精度加法，但是该算法的实现需要数组这一数据结构的实现，又让我联想到

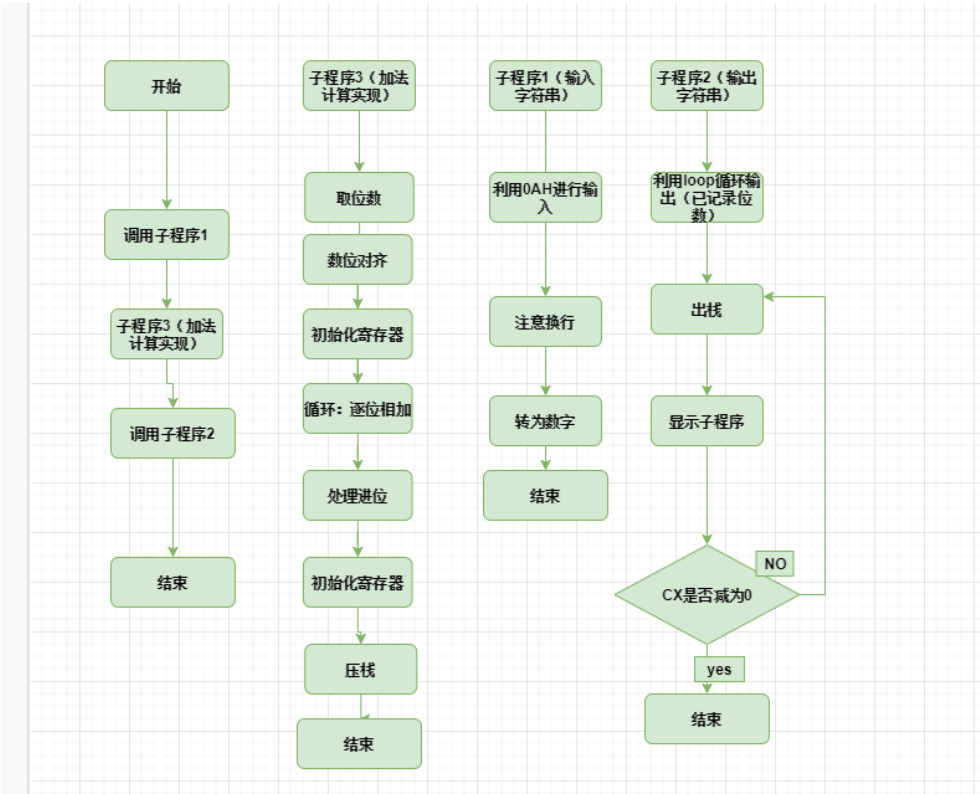
了七种寻址方式恰恰有我所需要的顺序存储结构，因此，我开始着手实现。

3. 输入 15 位的大数字，并相加.

画出流程图：



图表 16



图表 17

源代码：（第一次成功版）：

```

1. DATAS SEGMENT
2.     BUF1 DB 20      ;缓冲区 1: 最大长度 20
3.         DB ?        ;实际输入长度
4.         DB 20 DUP(?) ;存储输入的字符
5.     BUF2 DB 20      ;缓冲区 2: 最大长度 20
6.         DB ?        ;实际输入长度
7.         DB 20 DUP(?)
8.     max_len DB 1     ;存储两数中较长的位数
9.         DB ?        ;未使用
10.        DB 1 DUP(?)  ;实际存储 max_len
11.     min_len DB 1     ;存储两数中较短的位数
12.         DB ?        ;未使用
13.        DB 1 DUP(?)  ;实际存储 min_len
14. DATAS ENDS
15.
16. STACKS SEGMENT
17.     DB 200 DUP(?)   ;定义 200 字节的堆栈空间
18. STACKS ENDS
19.
20. CODES SEGMENT
21.     ASSUME CS:CODES, DS:DATAS, SS:STACKS

```

22.

23. **START:**

24. `MOV AX, DATAS` ; 初始化数据段

25. `MOV DS, AX`

26.

27. **CALL** INPUTS ; 调用子程序 1: 输入两个字符串

28. **CALL** ADD\_DOUBLE ; 调用子程序 3: 高精度加法

29.

30. ; 程序终止

31. `MOV AH, 4CH`

32. **INT** 21H

33.

34. INPUTS PROC ; 子程序 1: 输入两个字符串

35. `LEA DX, BUF1` ; 读取第一个字符串

36. `MOV AH, 0AH`

37. **INT** 21H

38.

39. ; 输出回车换行

40. `MOV DL, 0DH`

41. `MOV AH, 02H`

42. **INT** 21H

43. `MOV DL, 0AH`

44. MOV AH, 02H

45. INT 21H

46.

47. LEA DX, BUF2 ; 读取第二个字符串

48. MOV AH, 0AH

49. INT 21H

50. RET

51. INPUTS ENDP

52.

53. ADD\_DOUBLE PROC ; 子程序 3: 高精度加法 (存在逻辑错误)

54. LEA DI, [BUF1 + 2] ; DI 指向 BUF1 的第一个字符

55. LEA SI, [BUF2 + 2] ; SI 指向 BUF2 的第一个字符

56.

57. ; 计算 BUF1 的末尾地址 (从右到左处理)

58. MOV CL, [BUF1 + 1] ; CL = BUF1 的实际长度

59. MOV CH, 00H

60. ADD DI, CX ; DI 指向 BUF1 的最后一个字符

61. DEC DI ; 调整到正确位置

62. MOV max\_len, CL ; 初始化 max\_len 为 BUF1 长度

63.

64. ; 计算 BUF2 的末尾地址

65. MOV CL, [BUF2 + 1]

```

66.  MOV CH, 00H
67.  ADD SI, CX
68.  DEC SI
69.  MOV min_len, CL ; 初始化 min_len 为 BUF2 长度
70.
71.  ; 确定 max_len 和 min_len
72.  CMP CL, [BUF1 + 1]
73.  JBE NEXT0
74.  MOV CL, [BUF1 + 1]; 如果 BUF2 更长, 更新 max_len 和 min_len
75.  MOV max_len, CL
76.  MOV AL, [BUF2 + 1]
77.  MOV min_len, AL
78.
79. NEXT0:
80.  XOR AH, AH ; 清空 AH
81.  XOR DX, DX ; 清空 DX (用于存储进位)
82.
83. ADD_LOOP:
84.  ;
85.  MOV AL, [DI] ; 取 BUF1 的字符 (ASCII 码)
86.  ADD AL, [SI] ; 加上 BUF2 的字符 (ASCII 码)
    
```

```

87.  ADD AL, DL      ; 加上进位（错误：DL 是前一次的进位，但此时 AL 已经是
    错误值）
88.  AAA              ; 调整加法结果（因 AL 为 ASCII 码，AAA 无法正确调整）
89.
90.  XOR DX, DX      ; 清空 DX
91.  DEC DI          ; 前移指针
92.  DEC SI
93.  MOV DL, AH      ; 保存进位到 DL（AH 来自 AAA 调整）
94.  XOR AH, AH
95.  PUSH AX         ; 保存当前位结果（AL 为错误值）
96.  LOOP ADD_LOOP   ; 循环次数为 CX（min_len），剩余高位未处理
97.
98.  ; 【后续改进点】未处理较长数字的高位和最终进位
99.  XOR DH, DH
100. PUSH DX         ; 保存最后一次的进位
101.
102. ; 输出换行
103. NEXT1:
104.  MOV DL, 0DH
105.  MOV AH, 02H     ; 美化输出
106.  INT 21H
107.  MOV DL, 0AH     ; 美化输出
108.  MOV AH, 02H
    
```

```

109.  INT 21H
110.
111.  ; 输出结果
112.  MOV CL, max_len
113.  POP DX          ; 取出最高位进位
114.  CMP DL, 0H
115.  JZ NEXT2
116.  ADD DL, 30H      ; 转换为 ASCII
117.  MOV AH, 02H
118.  INT 21H
119.
120. NEXT2:
121. PRINT_LOOP:      ; 弹出并输出各位结果
122.  POP DX
123.  ADD DL, 30H      ; 转换为 ASCII
124.  MOV AH, 02H
125.  INT 21H
126.  LOOP PRINT_LOOP
127.  RET
128. ADD_DOUBLE ENDP
129.
130. CODES ENDS

```

### 131. END START

源代码（终极版）：（不同位数加法的实现）

```

1.
    DATAS SEGMENT                ; 数据段定义
2.     BUF1 DB 20                ; BUF1 缓冲区定义：最大长度 20 字节
3.     DB ?                      ; 存储实际输入字符数
4.     DB 20 DUP(?)             ; 存储输入的 20 个字符
5.     BUF2 DB 20                ; BUF2 缓冲区定义，结构同 BUF1
6.     DB ?
7.     DB 20 DUP(?)
8.     max_len DB 1              ; 存储较长字符串的长度
9.     DB ?                      ; 初始化占位符
10.    DB 1 DUP(?)               ; 实际存储空间
11.    min_len DB 1              ; 存储较短字符串的长度
12.    DB ?
13.    DB 1 DUP(?)
14. DATAS ENDS
15.
16. STACKS SEGMENT              ; 堆栈段定义
17.    DB 200 DUP(?)             ; 分配 200 字节堆栈空间
18. STACKS ENDS
19.

```

```

20. CODES SEGMENT                ; 代码段定义

21.  ASSUME CS:CODES,DS:DATAS,SS:STACKS ; 设定段寄存器关联

22.

23. START:

24.  MOV AX,DATAS                ; 初始化数据段寄存器

25.  MOV DS,AX

26.

27.  CALL INPUTS                ; 调用输入子程序获取两个字符串

28.  CALL ADD_DOUBLE            ; 调用高精度加法子程序

29.

30.  MOV AH,4CH                 ; DOS 程序终止功能号

31.  INT 21H                    ; 返回 DOS 系统

32.

33. ; 输入子程序

34. INPUTS PROC

35.  LEA DX,BUF1                ; 加载 BUF1 结构地址

36.  MOV AH,0AH                 ; DOS 缓冲输入功能

37.  INT 21H                    ; 读取第一个字符串

38.

39.  MOV DL, 0DH                ; 准备回车符

40.  MOV AH, 02H                ; DOS 字符输出功能

41.  INT 21H                    ; 输出回车
    
```

42.

43. MOV DL, 0AH ; 准备换行符

44. MOV AH, 02H

45. INT 21H ; 输出换行

46.

47. MOV DX, OFFSET BUF2 ; 加载 BUF2 结构地址

48. MOV AH, 0AH

49. INT 21H ; 读取第二个字符串

50. RET

51. INPUTS ENDP

52.

53. ; 高精度加法子程序

54. ADD\_DOUBLE PROC

55. LEA DI, [BUF1 + 2] ; DI 指向 BUF1 字符起始地址

56. LEA SI, [BUF2 + 2] ; SI 指向 BUF2 字符起始地址

57.

58. MOV CL, [BUF2 + 1] ; 取 BUF2 实际长度

59. MOV min\_len, CL ; 存储为 min\_len

60. MOV CL, [BUF1 + 1] ; 取 BUF1 实际长度

61. MOV max\_len, CL ; 存储为 max\_len

62.

63. CMP CL, [BUF2 + 1] ; 比较两字符串长度

```

64.  JAE NEXT3                ; BUF1 较长则跳转
65.  ; 若 BUF2 较长, 交换指针和长度值
66.  LEA DI,[BUF2 + 2]        ; DI 指向较长字符串
67.  LEA SI,[BUF1 + 2]
68.  MOV AL,[BUF2 + 1]
69.  MOV max_len,AL           ; 更新 max_len
70.  MOV AL,[BUF1 + 1]
71.  MOV min_len,AL           ; 更新 min_len
72.
73. NEXT3:
74.  XOR AH,AH
75.  MOV AL,max_len
76.  ADD DI,AX                 ; DI 移到较长字符串末尾
77.  DEC DI                    ; 调整到最后一个字符
78.
79.  XOR AH,AH
80.  MOV AL,min_len
81.  ADD SI,AX                 ; SI 移到较短字符串末尾
82.  DEC SI                    ; 调整到最后一个字符
83.
84.  MOV CL,min_len            ; 设置循环次数为短串长度
85.  XOR CH,CH
    
```

```

86.  XOR DX,DX                ; 清空进位寄存器
87.
88.  ADD_LOOP:
89.  MOV AL,[DI]              ; 取长串当前字符
90.  XOR AH,AH
91.  ADD AL,[SI]              ; 加短串当前字符
92.  ADD AL,DL                ; 加上次进位值
93.  AAA                      ; ASCII 加法调整
94.  XOR DX,DX                ; 清空进位
95.  DEC DI                   ; 前移长串指针
96.  DEC SI                   ; 前移短串指针
97.  MOV DL,AH                ; 保存新进位值
98.  XOR AH,AH
99.  PUSH AX                  ; 当前位结果压栈
100. LOOP ADD_LOOP           ; 循环处理短串长度次数
101.
102.  MOV CL,max_len
103.  SUB CL,min_len          ; 计算剩余位数
104.  CMP CL,0                ; 判断是否有剩余位
105.  JE NEXT1                ; 无剩余跳转输出
106.
107.  ADD_REMAIN:
    
```

```

108.  MOV AL,[DI]           ;取剩余位字符
109.  XOR AH,AH
110.  ADD AL,DL             ;加进位值
111.  AAA                   ;调整结果
112.  XOR DX,DX
113.  DEC DI                ;前移指针
114.  MOV DL,AH             ;保存新进位
115.  XOR AH,AH
116.  PUSH AX               ;当前位结果压栈
117.  LOOP ADD_REMAIN       ;处理剩余位数
118.
119.  PUSH DX               ;最后进位值压栈
120.
121. NEXT1:
122.  ; 输出处理
123.  MOV DL,0DH            ;输出回车
124.  MOV AH,02H
125.  INT 21H
126.
127.  MOV DL,0AH            ;输出换行
128.  MOV AH,02H
129.  INT 21H

```

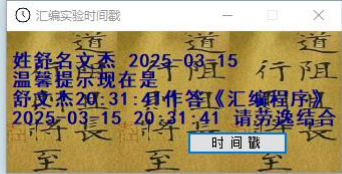
```

130.
131.  MOV CL,max_len          ; 获取结果位数
132.  POP DX                  ; 弹出最高位进位
133.  CMP DL,0                ; 判断是否有进位
134.  JZ NEXT2                ; 无进位跳转
135.  ADD DL,30H              ; 进位转 ASCII
136.  MOV AH,02H
137.  INT 21H                ; 输出最高位进位
138.
139. NEXT2:
140. PRINT_LOOP:
141.  POP DX                  ; 弹出一位结果
142.  ADD DL,30H              ; 转 ASCII 字符
143.  MOV AH,02H
144.  INT 21H                ; 输出该位
145.  LOOP PRINT_LOOP         ; 循环输出所有位
146.  RET
147. ADD_DOUBLE ENDP
148.
149. CODES ENDS
150.  END START              ; 程序结束

```

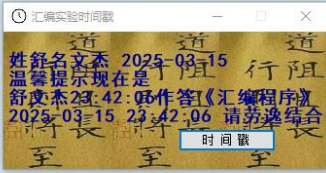
1、用 windows 集成编译环境编写. ASM 源程序。

```
01 ;默认采用ML6.11汇编程序
02 DATAS SEGMENT
03 ;此处输入数据段代码
04 DATAS ENDS
05
06 STACKS SEGMENT
07 ;此处输入堆栈段代码
08 STACKS ENDS
09
10 CODES SEGMENT
11     ASSUME CS:CODES,DS:DATAS,SS:STACKS
12 START:
13     MOV AX,DATAS
14     MOV DS,AX
15     ;此处输入代码段代码
16     MOV AH,4CH
17     INT 21H
18 CODES ENDS
19     END START
```



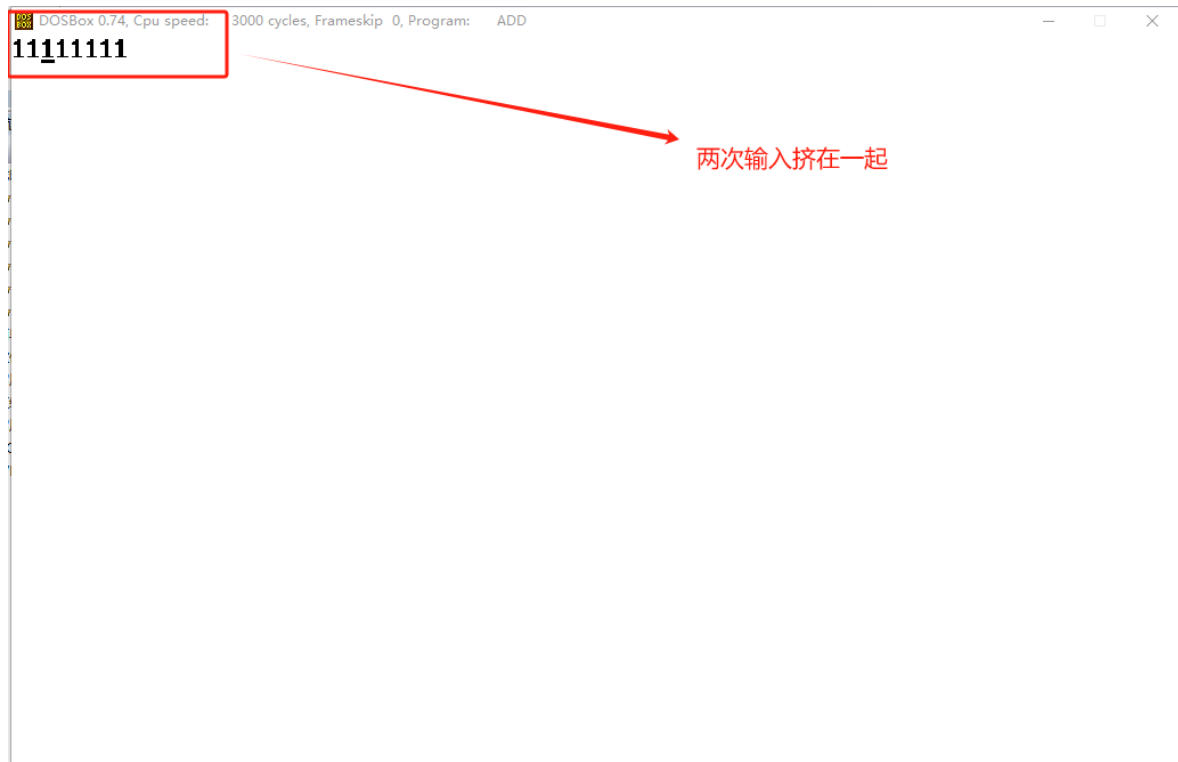
图表 18

```
001 DATAS SEGMENT
002     BUF1 DB 20
003         DB ?
004     DB 20 DUP(?)
005     BUF2 DB 20
006         DB ?
007     DB 20 DUP(?)
008
009 DATAS ENDS
010
011 STACKS SEGMENT
012     DB 200 DUP(?)
013 STACKS ENDS
014
015 CODES SEGMENT
016     ASSUME CS:CODES,DS:DATAS,SS:STACKS
017 START:
018     MOV AX,DATAS
019     MOV DS,AX
020
021     CALL INPUTS
022     CALL ADD_DOUBLE
023
024     ;MOV CL,[BUF2 + 1]
025     ;MOV CH,00H
026     ;CMP CL,[BUF1 + 1]
027     ;JAE NEXT3
028     ;MOV CL,[BUF1 + 1]
029 ;NEXT3:
030 ;PRINT_LOOP:
031     ;POP DX
032     ;ADD DL,30H
033     ;MOV AH,02H
034     ;INT 21H
035     ;LOOP PRINT_LOOP
036
```



图表 19

很好，我们执行一下：



这是为什么呢？原来，我没有加回车符与换行隔开输入。

```

13 STACKS ENDS
14
15 CODES SEGMENT
16     ASSUME CS:CODES,DS:DATAS,SS:STACKS
17 START:
18     MOV AX,DATAS
19     MOV DS,AX
20
21     CALL INPUTS
22
23     MOV AH,4CH
24     INT 21H
25
26 INPUTS PROC
27     LEA DX,BUF1
28     MOV AH,0AH
29     INT 21H
30
31     MOV DL,0DH ; 回车符
32     MOV AH,02H
33     INT 21H
34
35     MOV DL,0AH ; 换行符
36     MOV AH,02H
37     INT 21H
38
39     MOV DX,OFFSET BUF2
40     MOV AH,0AH
41     INT 21H
42     RET
    
```

隔开输入

再次运行：

[illegible]

这是为什么呢？怎么会执行这么多次？我终于想起来前几次实验在中断后会压入标志位，难道我把标志位给弄不见了？

```

DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG

AX=0001 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0017 DI=0001
DS=079C ES=078C SS=079B CS=07AC IP=007C  NU UP EI PL ZR NA PE NC
07AC:007C 50          PUSH  AX
-t

AX=0001 BX=0000 CX=0001 DX=0000 SP=FFFC BP=0000 SI=0017 DI=0001
DS=079C ES=078C SS=079B CS=07AC IP=007D  NU UP EI PL ZR NA PE NC
07AC:007D E2EC      LOOP   006B
-t

AX=0001 BX=0000 CX=0000 DX=0000 SP=FFFC BP=0000 SI=0017 DI=0001
DS=079C ES=078C SS=079B CS=07AC IP=007F  NU UP EI PL ZR NA PE NC
07AC:007F 52          PUSH  DX
-t

AX=0001 BX=0000 CX=0000 DX=0000 SP=FFFA BP=0000 SI=0017 DI=0001
DS=079C ES=078C SS=079B CS=07AC IP=0080  NU UP EI PL ZR NA PE NC
07AC:0080 C3          RET
-t

AX=0001 BX=0000 CX=0000 DX=0000 SP=FFFC BP=0000 SI=0017 DI=0001
DS=079C ES=078C SS=079B CS=07AC IP=0000  NU UP EI PL ZR NA PE NC
07AC:0000 B89C07     MOV   AX,079C
    
```

怎么跳回去了

果然，看来真是这样，

```

;OUTPUTS PROC          ;子程序2 (cx的保护需要注意) (写到子程序去)
;MOV CL,[BUF2 + 1]
;CMP CL,[BUF1 + 1]
;JAE NEXT3
;MOV CL,[BUF1 + 1]
;NEXT3:
;PRINT_LOOP:
;POP DX
;ADD DL,30H
;MOV AH,02H
;INT 21H
;LOOP PRINT_loop
;RET
;OUTPUTS ENDP
    
```

这里弹栈的时候把标志位也给弹出去，或者没弹，导致出子程序出现了故障

于是，我把这部分与子程序 3 结合了，我学会了栈平衡，不要对栈只出不入。

## 2.运行查 BUG



图表 20



图表 21

在完成第一次成功版时，我考虑到了可能由于不同长度的加法，，需要我们在过程中进行数位对齐的操作，但是当时没有将其付诸实践，后来发现这样的操作有太过麻烦之嫌，于是我将一次加法拆分了，分为高位加法与低位加法，以一个固定的指针指向高数位的数，并将二者数位指向最小端，最后同时经过两次加法，完成了该算法。

【实验心得】

1. AAA 指令的救赎

开始时我为如何进行进位操作而发愁，因为如果进行判断操作会使程序变得非常臃肿，后来引入的 AAA 指令成了转折点——它可将 AL 中的二进制和自动拆分为十位（AH）和个位（AL），比如 0x0A 变为 0x0100。这简直将我的表达精简了不少，只是，我需要注意的是 AAA 指令只对 AX 寄存器有效，使用之前需要初始化寄存器。

## 2. 子程序：从混沌到秩序的重构

把所有功能堆砌在主程序里的初期版本，调试时就像在迷宫找出口。把输入转换和十进制输出拆成 TRANS、TTO10 子程序后，代码顿时清爽。有次测试 TTO10 时，手动给 AL 赋值 0x0F 却输出乱码，发现漏了 ADD DL,30H——这让我意识到：模块化不仅是拆分，更要保证每个模块自治。后来添加十六进制输出功能时，直接复用 TRANS 子程序节省了 70% 的时间，验证了模块化的扩展价值。

## 3. 栈平衡：精确到指令的数学题

16 转 10 进制与高精度加法计算时，我曾因 PUSH/POP 次数不匹配导致栈指针错乱。最惨痛的一次是处理 15 位加法，循环结束后 SP 偏移了 4 字节，程序直接跳转到数据段执行。后来画出栈变化示意图才发现：每个 PUSH 必须对应一个 POP，就像括号必须成对闭合。现在我会在循环入口记录栈深度，如同会计对账般严格核对进出次数。

## 4. 输出格式化：看不见的用户体验

初期忽视 0DH/0AH 换行的后果，是输入提示和结果挤成扭曲的一行文字。加上换行后，程序顿时有了“专业感”。但很快发现新问题：输出十位数后忘记恢复 AX 寄存器，导致个位数显示成乱码。这教会我：输出不仅是功能实现，更是与用户的无声对话，每个字符的位置都需精心设计。

## 5. 寄存器芭蕾：在细节处起舞

忘记清空 AH 寄存器导致的高位数据残留，DI/SI 指针越界引发的内存错误，这些教训让我养成条件反射般的习惯：关键操作前必先 XOR 清零，移动指针必先校验偏移

量。有次调试时连续记录 200 条指令的寄存器状态，最终揪出某个循环漏掉的 DX 清零——这种经历让我真正理解，汇编编程是与 CPU 的精确共舞，每个动作都要落在时钟周期的节拍上。



## 知识点

### 1. EQU 与 ' =' 的区别.

在 8086 汇编语言中，EQU 和等号 (=) 的核心区别在于符号的不可变性与灵活性。EQU 用于定义不可修改的常量，其值在预处理阶段被固定且全局替换，支持数值、字符串、表达式甚至代码片段（类似宏），适合定义硬件端口地址、ASCII 字符、固定公式等永久性常量；而等号 (=) 本质是符号赋值，允许重复定义，每次引用时按最新赋值动态计算，仅支持数值或简单表达式，常用于临时变量、循环计数或结构体偏移量的动态计算。EQU 强调编译前的确定性（如 `PORT EQU 80H` 一经定义不可更改），而等号（如 `COUNT = 10` 可后续修改）更适用于需要多次调整的场景，两者分别对应“常量定义”与“变量赋值”的逻辑需求，需根据符号是否需要重用或变化来选择。

### 2.\$的各种用法

#### 1.当前位置计数器

\$代表汇编器当前正在处理的内存地址（即代码或数据段的当前位置）。这一特性常用于计算变量或缓冲区的长度，例如通过\$ - 标签的形式直接得出数据占用的字节数。此外，在代码段中，\$可配合跳转指令实现动态地址计算，例如 `JMP $ + 5` 会跳转到当前指令地址后 5 字节的位置，适用于自修改代码或特定内存布局需求。

#### 2.DOS 字符串终止符

在调用 DOS 系统功能（如 INT 21H 的 09H 号显示字符串功能）时，\$被约定为字符串的结束标志（ASCII 码为 24H）。此时，字符串必须以\$结尾，否则中断调用可能无法正确识别字符串边界。需注意，尽管部分现代汇编器不强制要求此符号，但 DOS 环境的系统功能仍依赖这一传统约定。

### 3. 程序流程控制

**JMP \$**是一种常见的无限循环写法，通过跳转到当前地址使程序暂停执行。此用法常用于等待外部事件（如用户输入或硬件中断），或在调试阶段临时挂起程序。例如，在操作系统引导程序或中断服务例程中，可用此指令保持程序处于待命状态。

### 4. 内存布局与结构体偏移

结合 **ORG** 指令设置内存起始地址时，**\$**可用于动态计算结构体字段或数组的偏移量。例如，通过 **MOV AX, \$**可获取当前代码位置，辅助实现地址相关的动态计算，尤其在手动管理内存分布的场景中十分实用。

## 3. ORG 指令

**ORG**（起始地址伪指令）用于明确程序或数据在内存中的起始位置。它直接影响汇编器对标签、变量和跳转指令的地址计算。例如，在 DOS 的 COM 文件中，程序必须从 **0x100** 地址加载，需用 **ORG 100H** 声明；引导扇区代码需设置 **ORG 7C00H** 以匹配 BIOS 加载地址。未正确设置 **ORG** 会导致地址错位，引发程序崩溃或逻辑错误。

作用域限制：**ORG** 仅作用于当前段，多段程序需分别设置起始地址。

兼容性差异：不同汇编器（如 NASM/MASM）对 **ORG** 的支持范围不同，需查阅文档。

**ORG** 是汇编语言中管理内存布局的关键工具，尤其适用于需精确控制加载地址的场景（如系统级开发）。正确使用可确保程序与硬件环境无缝衔接，避免地址错位风险，是底层编程不可或缺的指令之一。